

PROJECT REPORT
on
SMART PARKING MANAGEMENT
SYSTEM
(SmartParking)

By

Narendra Balunda - 31

Naresh Chandra Kumawat - 32

School Of Information Technology(SOIT)

MSC(CS)

--

ACKNOWLEDGEMENT

We take great pleasure in presenting our project titled "*Smart Parking Management System (SmartParking)*", completed as part of the academic curriculum for the degree of MSC(CS).

We would like to express our sincere gratitude to our respected guide for their valuable guidance, continuous support, and encouragement throughout the development of this project. Their insightful suggestions and constructive feedback helped us in successfully completing this work.

We are also thankful to all the faculty members of the department for providing us with the necessary knowledge and resources required for the completion of this project.

We extend our heartfelt thanks to our friends and classmates for their support, cooperation, and valuable suggestions during the project development phase.

Finally, we would like to thank our family members for their constant motivation and encouragement, which helped us complete this project successfully.

This project has helped us gain practical knowledge in web technologies, database management, backend API development, JWT-based authentication, and real-time email notification systems, and has been a valuable learning experience for us.

Project Team

Smart Parking Management System (SmartParking)

MSC(CS)

Academic Year: 2025–26

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Existing System and Need for System	4
1.2 Scope of Work	5
1.3 Operating Environment – Hardware and Software	6

CHAPTER 2: PROPOSED SYSTEM

2.1 Proposed System	7
2.2 Objectives of System	8
2.3 User Requirements	9

CHAPTER 3: ANALYSIS & DESIGN

3.1 Data Flow Diagram (DFD)	10
3.2 Functional Decomposition Diagram (FDD)	11
3.3 UML Diagrams	12
3.3.1 Class Diagram	12
3.3.2 Object Diagram	13
3.3.3 Use Case Diagram	14
3.3.4 Sequence Diagram	15
3.3.5 Collaboration Diagram	16
3.3.6 State Diagram	17
3.3.7 Activity Diagram	18
3.3.8 Component Diagram	19
3.3.9 Deployment Diagram	20
3.4 Entity Relationship Diagram (ERD)	21
3.5 Data Dictionary	22
3.6 Table Design	23
3.7 Code Design	24

CHAPTER 4: USER MANUAL

4.1 Operations Manual / Menu Explanation	26
4.2 Menu Screens	27
4.3 Project Code	28

CHAPTER 5: TESTING

5.1 Testing Test Plan	32
5.2 Black Box Testing or Data Validation	33
5.3 White Box Testing or Functional Validation	34

CHAPTER 6: LIMITATIONS AND ENHANCEMENT

6.1 Drawbacks and Limitations	37
6.2 Proposed Enhancements	38
6.3 Conclusions	39
6.4 Bibliography	40

CHAPTER 1: INTRODUCTION

1.1 Existing System and Need for System

In modern urban environments, parking management has become one of the most pressing challenges faced by city dwellers, shopping malls, hospitals, and educational institutions. The conventional approach to parking relies entirely on manual operations — security guards directing vehicles, physical token systems, and paper-based records — all of which are inherently inefficient, error-prone, and unable to scale with rising vehicle populations.

Traditional parking systems suffer from numerous shortcomings. Drivers must physically explore an entire parking facility looking for a vacant spot, wasting significant fuel and time. There is no centralized visibility into slot availability, no digital record of active bookings, and no mechanism for prior reservation. Disputes over occupied slots are common, and revenue leakage due to untracked entries is significant.

Furthermore, the COVID-19 pandemic accelerated the demand for contactless, digital-first solutions across all sectors. Parking is no exception. The need for a system that allows users to browse available parking spots, make reservations in advance, receive real-time email confirmations, and manage their bookings entirely from a web browser is more urgent than ever.

The SmartParking Management System was conceived to directly address all these pain points. It replaces fragmented, manual workflows with a fully integrated, web-based platform that connects three key actors: Parking Hosts (who list and manage their spots), Users (who search, book, and cancel reservations), and Administrators (who oversee platform-wide operations). Every significant action — registration, login, booking, and cancellation — is accompanied by an automated email notification, ensuring transparency and trust at every step.

1.2 Scope of Work

The scope of the SmartParking project encompasses the design, development, testing, and deployment of a full-stack web application for intelligent parking spot management. The system is built using ASP.NET Core 8.0 for the backend REST API, a vanilla HTML/CSS/JavaScript frontend, and MySQL as the persistent relational database.

The key functional areas covered within this project are:

- **User Authentication:** Secure registration and login using JWT (JSON Web Tokens) with role-based access control (User, Host, Admin).

- **Parking Spot Management:** Hosts can list, update, and remove parking spots with details such as name, address, GPS coordinates, price per hour, and total capacity.
- **Booking System:** Registered users can search for available spots, select time slots, add their vehicles, and confirm reservations. The system enforces conflict detection to prevent double-booking.
- **Email Notification Service:** An integrated SMTP email service using Gmail dispatches automated confirmations for registration, login alerts, booking confirmation, cancellation, and password reset.
- **Forgot Password Flow:** A secure, token-based password reset mechanism generates a 6-digit OTP with a 15-minute expiry, delivered via email, allowing users to reset their passwords safely.
- **Vehicle Management:** Users can register their vehicles and associate them with bookings.
- **Admin Dashboard:** An administrative interface for platform management.
- **Camera Integration:** A camera controller supports entry/exit image capture endpoints for advanced tracking.

The scope explicitly excludes payment gateway integration, mobile application development, and IoT-based physical barrier automation, which are planned for future enhancements.

1.3 Operating Environment – Hardware and Software

Hardware Requirements

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 or above
RAM	4 GB	8 GB or above
Storage	20 GB HDD	50 GB SSD
Network	Broadband Internet	Stable 10 Mbps+
Display	1024×768 resolution	1920×1080 Full HD

Software Requirements

Component	Technology / Version
Operating System	Windows 10/11, Linux, or macOS
Backend Framework	ASP.NET Core 8.0 (.NET 8 SDK)
Frontend	HTML5, CSS3, Vanilla JavaScript (ES6+)

Database	MySQL 8.0 (via Pomelo EF Core provider)
ORM	Entity Framework Core 8.0
Authentication	JWT Bearer Tokens (HS256)
Email Service	SMTP via Gmail (System.Net.Mail)
API Documentation	Swagger / OpenAPI 3.0
Web Browser	Google Chrome 110+ / Microsoft Edge
IDE	Microsoft Visual Studio 2026 / VS Code

CHAPTER 2: PROPOSED SYSTEM

2.1 Proposed System

The SmartParking Management System is proposed as a comprehensive, cloud-ready, full-stack web application designed to digitalize and automate all aspects of parking spot management. The system operates on a client-server architecture where a RESTful API backend (ASP.NET Core) serves a Single Page Application (SPA) frontend (HTML/JS) through static file serving.

The system is structured around three distinct roles:

- **Admin:** Has unrestricted access to the entire platform. Can manage users, monitor all bookings, and oversee parking spot listings.
- **Host:** Can register their parking facilities, list available spots with pricing and capacity, and view bookings made at their locations.
- **User:** Can search for available parking spots, make time-based reservations, register vehicles, cancel bookings, and reset their password securely.

The proposed system replaces all manual record-keeping with a persistent MySQL database managed through Entity Framework Core. All user sessions are secured via JWT tokens issued at login, with token expiry of 24 hours. All critical events trigger automated email notifications through the integrated Gmail SMTP service, providing an audit trail and improving user trust.

The system architecture follows the MVC (Model-View-Controller) pattern on the backend. The frontend communicates exclusively through RESTful JSON APIs, making the system easily extensible to mobile applications in the future.

2.2 Objectives of System

The primary objectives of the SmartParking Management System are as follows:

1. **Digitalize Parking Management:** Eliminate manual, paper-based processes by providing a fully digital platform for listing, searching, booking, and managing parking spots.
2. **Prevent Double Booking:** Implement robust conflict detection that checks for overlapping reservations before confirming any new booking, ensuring data integrity.

3. **Secure Authentication:** Provide a robust, token-based authentication system with role-based access control to protect sensitive data and restrict unauthorized actions.
4. **Real-time Email Notifications:** Automatically notify users via email at every key interaction — account registration, login, booking confirmation, booking cancellation, and password reset.
5. **Secure Password Recovery:** Implement a safe, time-limited (15-minute OTP) forgot password flow that delivers a 6-digit confirmation code to the user's registered email.
6. **Vehicle Registration:** Allow users to associate vehicles with their profiles, enabling vehicle-specific parking reservations.
7. **Multi-role Dashboard:** Provide role-appropriate dashboards for Users (booking history), Hosts (spot and revenue management), and Admins (platform oversight).
8. **Scalability:** Build on a robust, industry-standard technology stack (ASP.NET Core + MySQL + EF Core) that can scale to handle thousands of concurrent users.

2.3 User Requirements

Functional Requirements

Req. No.	Requirement	User Role
FR-01	User shall be able to register with Name, Email, Phone, Password, and Role.	All
FR-02	System shall send a Welcome email upon successful registration.	All
FR-03	User shall be able to log in using email and password to receive a JWT token.	All
FR-04	System shall send a Login Alert email upon every successful login.	All
FR-05	Host shall be able to add/update/delete parking spots.	Host
FR-06	User shall be able to search for available parking spots by name or address.	User
FR-07	User shall be able to book a parking spot for a specific time range.	User
FR-08	System shall validate for conflicting bookings before confirming.	User
FR-09	System shall send a Booking Confirmation email upon successful booking.	User
FR-10	User shall be able to cancel an active booking.	User
FR-11	System shall send a Booking Cancellation email upon cancellation.	User
FR-12	User shall be able to initiate a Forgot Password flow via email OTP.	All
FR-13	System shall generate a 6-digit OTP valid for 15 minutes for password reset.	All
FR-14	User shall be able to add vehicles to their profile.	User

Non-Functional Requirements

Req. No.	Requirement
NFR-01	The system shall respond to any API call within 2 seconds under normal load.
NFR-02	All passwords must meet strong complexity requirements (8+ chars, upper, lower, digit, special).
NFR-03	JWT tokens shall expire after 24 hours to ensure session security.
NFR-04	The system shall be accessible on all modern browsers (Chrome, Edge, Firefox).
NFR-05	The UI shall be fully responsive and usable on desktop screens.
NFR-06	All API endpoints shall be documented via Swagger/OpenAPI.
NFR-07	SMTP email delivery shall not block the main API response thread (fire-and-forget).

CHAPTER 3: ANALYSIS & DESIGN

3.1 Data Flow Diagram (DFD)

The DFD illustrates the flow of data between users, the SmartParking system, and the database. At Level 0, the system accepts inputs from three actors: User, Host, and Admin, and produces outputs such as booking confirmations, email alerts, and dashboard data.

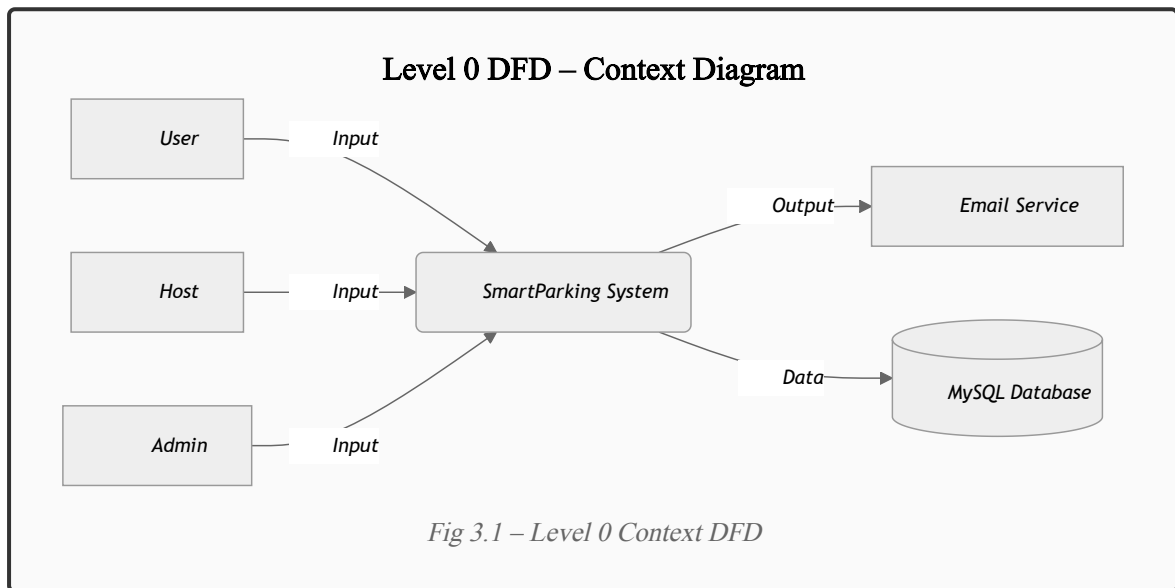


Fig 3.1 – Level 0 Context DFD

Level 1 DFD processes:

- P1: User Authentication (Register / Login / Forgot Password)
- P2: Parking Spot Management (Add / Update / Delete Spots)
- P3: Booking Management (Create / View / Cancel Bookings)
- P4: Vehicle Management (Add / View Vehicles)
- P5: Email Notification Dispatch

3.2 Functional Decomposition Diagram (FDD)

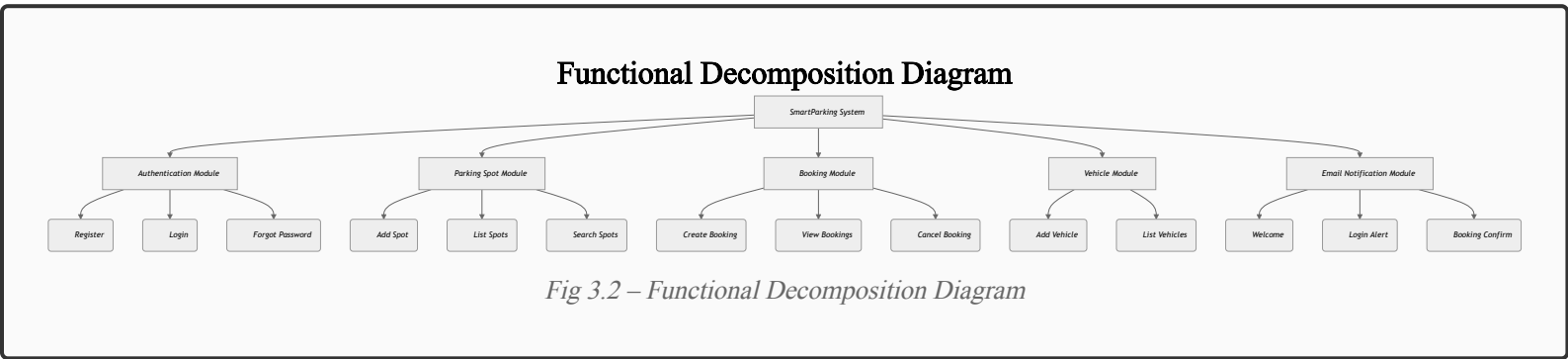
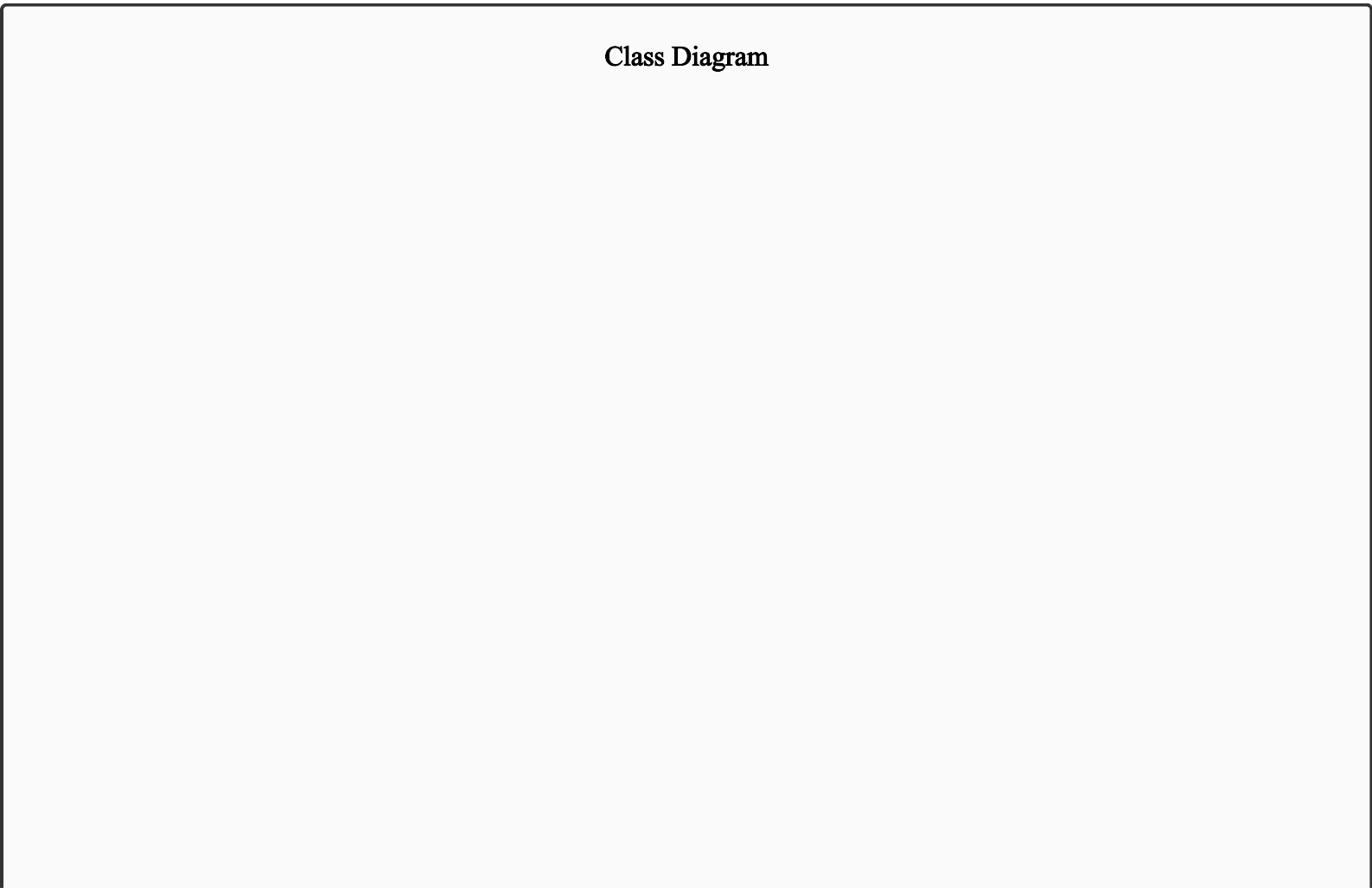


Fig 3.2 – Functional Decomposition Diagram

3.3 UML Diagrams

3.3.1 Class Diagram

The class diagram shows the four core domain models and their relationships.



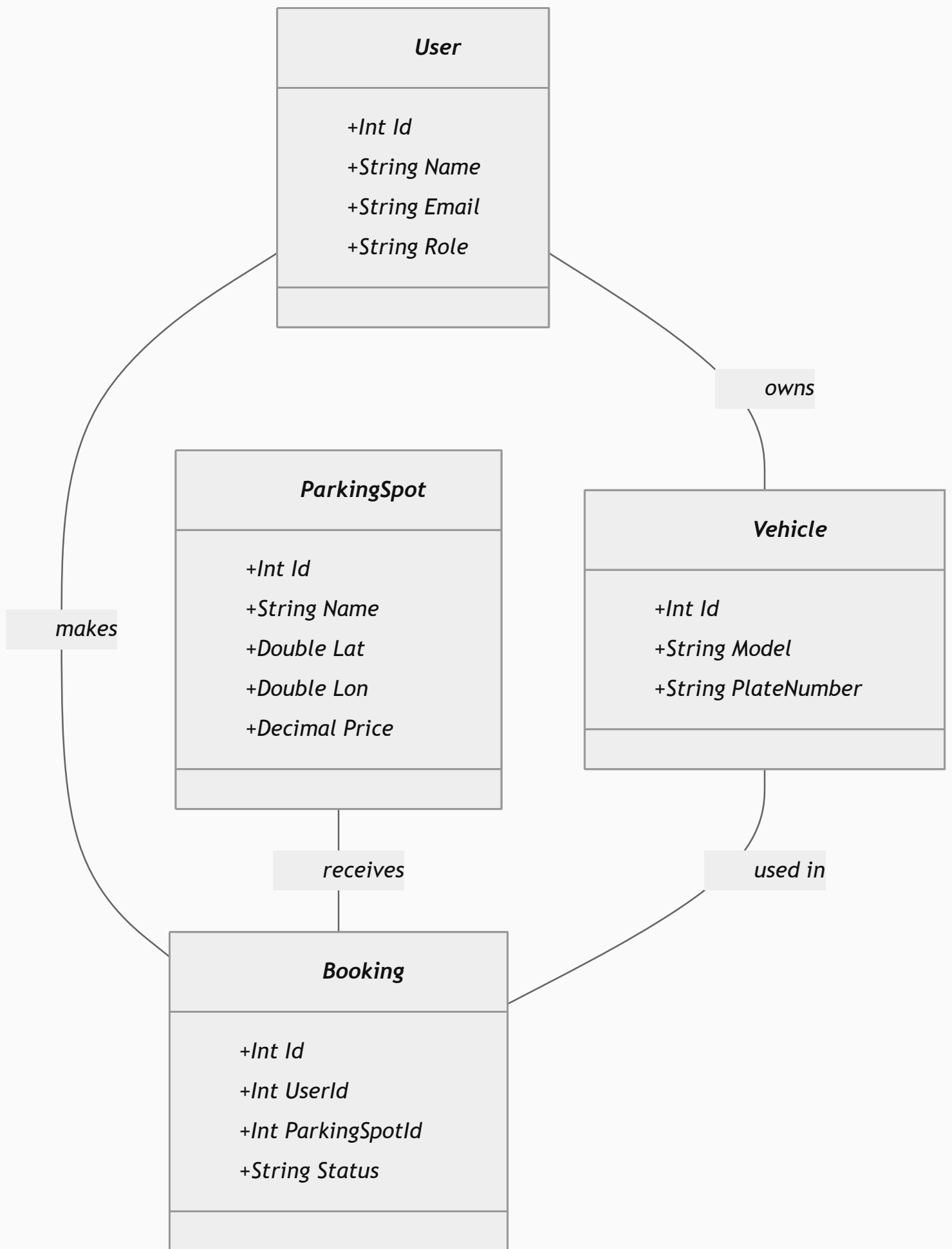


Fig 3.3.1 – Class Diagram

Fig 3.3.1 – Class Diagram

3.3.2 Object Diagram

Object Diagram

naresh : User

Id = 5

Name = "Naresh"

Role = "User"



booking1 : Booking

Id = 1

UserId = 5

SpotId = 2

Status = "Confirmed"



premiumSpot : ParkingSpot

Id = 2

Name = "Premium Spot"

Price = 10

Fig 3.3.2 – Object Diagram (Sample Instance)

Fig 3.3.2 – Object Diagram (Sample Instance)

3.3.3 Use Case Diagram

Use Case Diagram

SmartParking System

View Booking History

Book Parking Slot

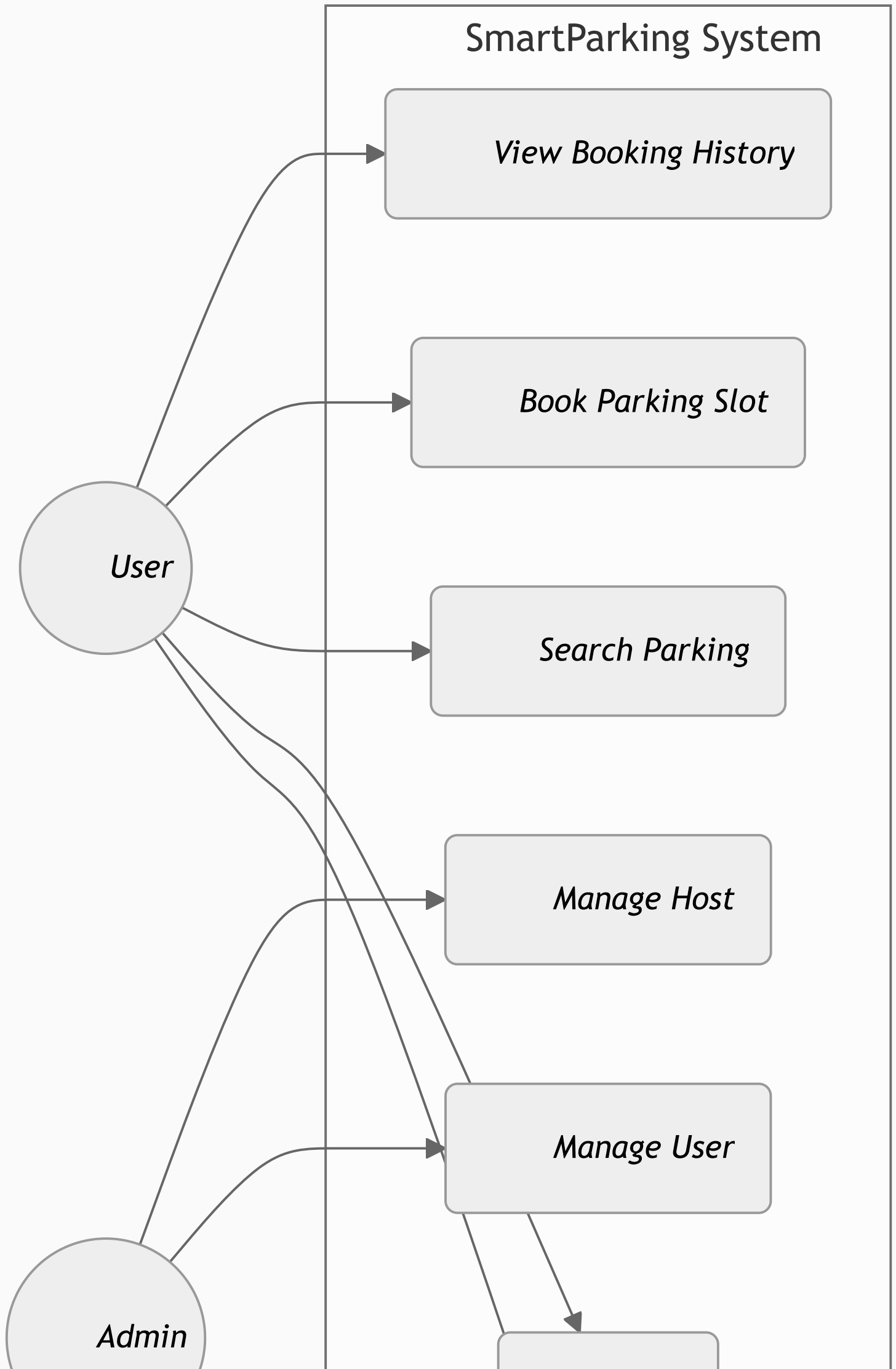
Search Parking

Manage Host

Manage User

User

Admin



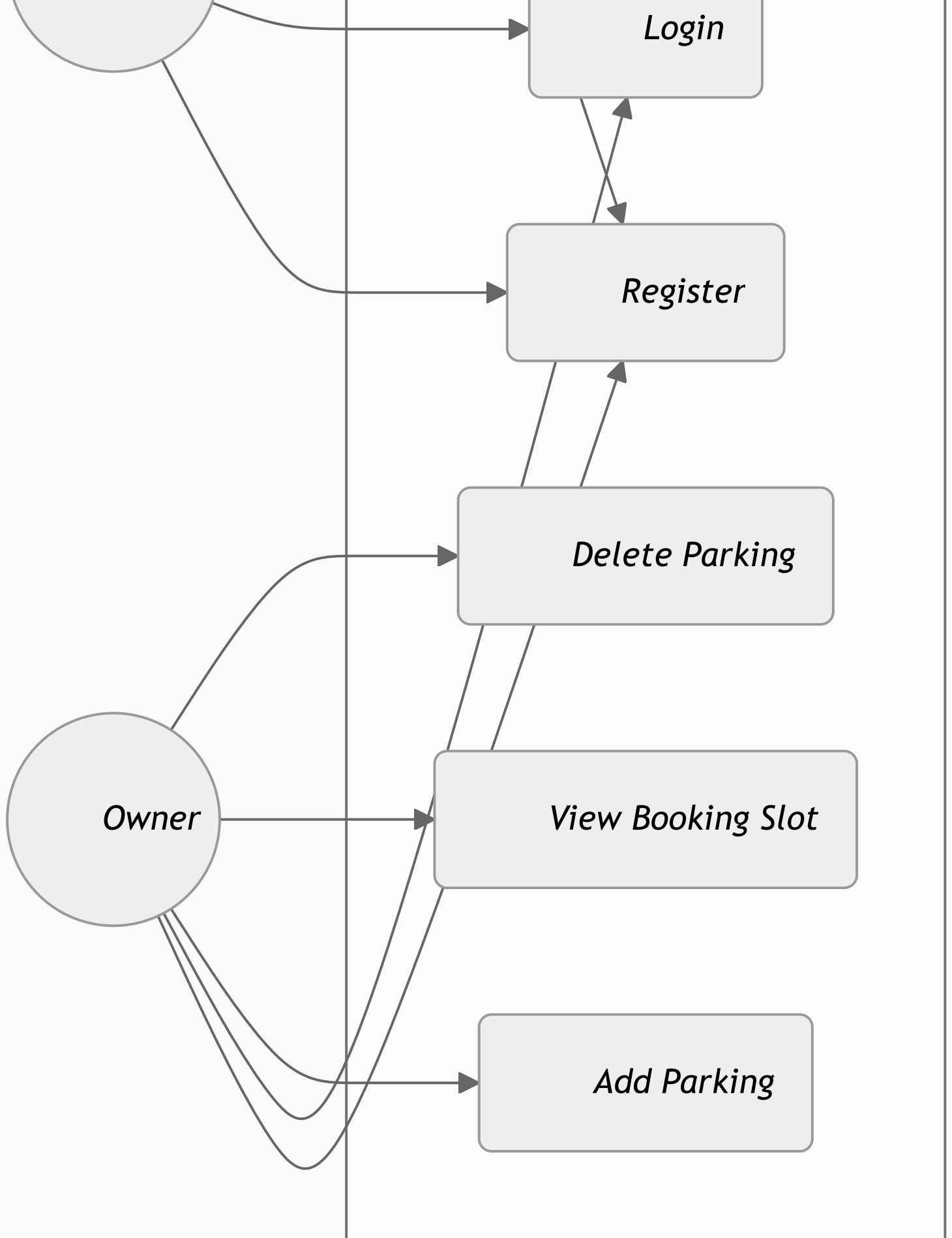


Fig 3.3.3 – Use Case Diagram

3.3.4 Sequence Diagram

Booking Flow Sequence:

Sequence Diagram (Booking Flow)

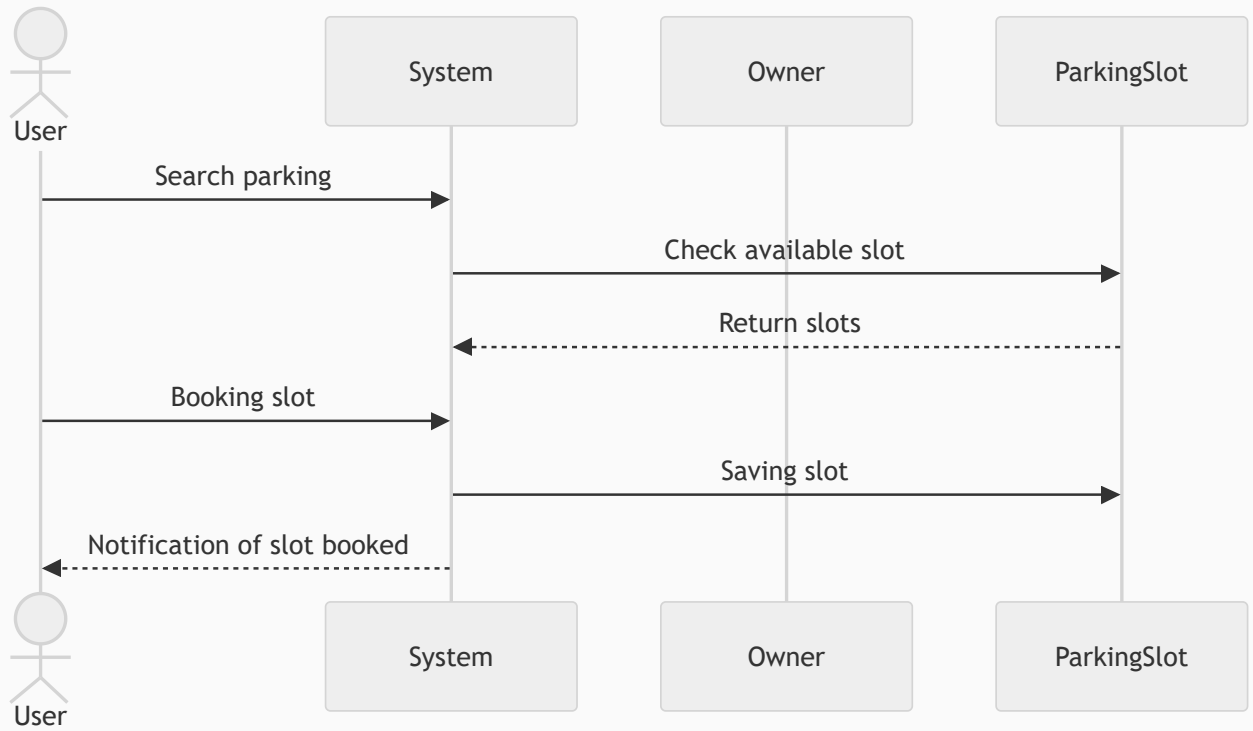


Fig 3.3.4 – Booking Sequence Diagram

3.3.5 Collaboration Diagram

Collaboration Diagram (Registration)

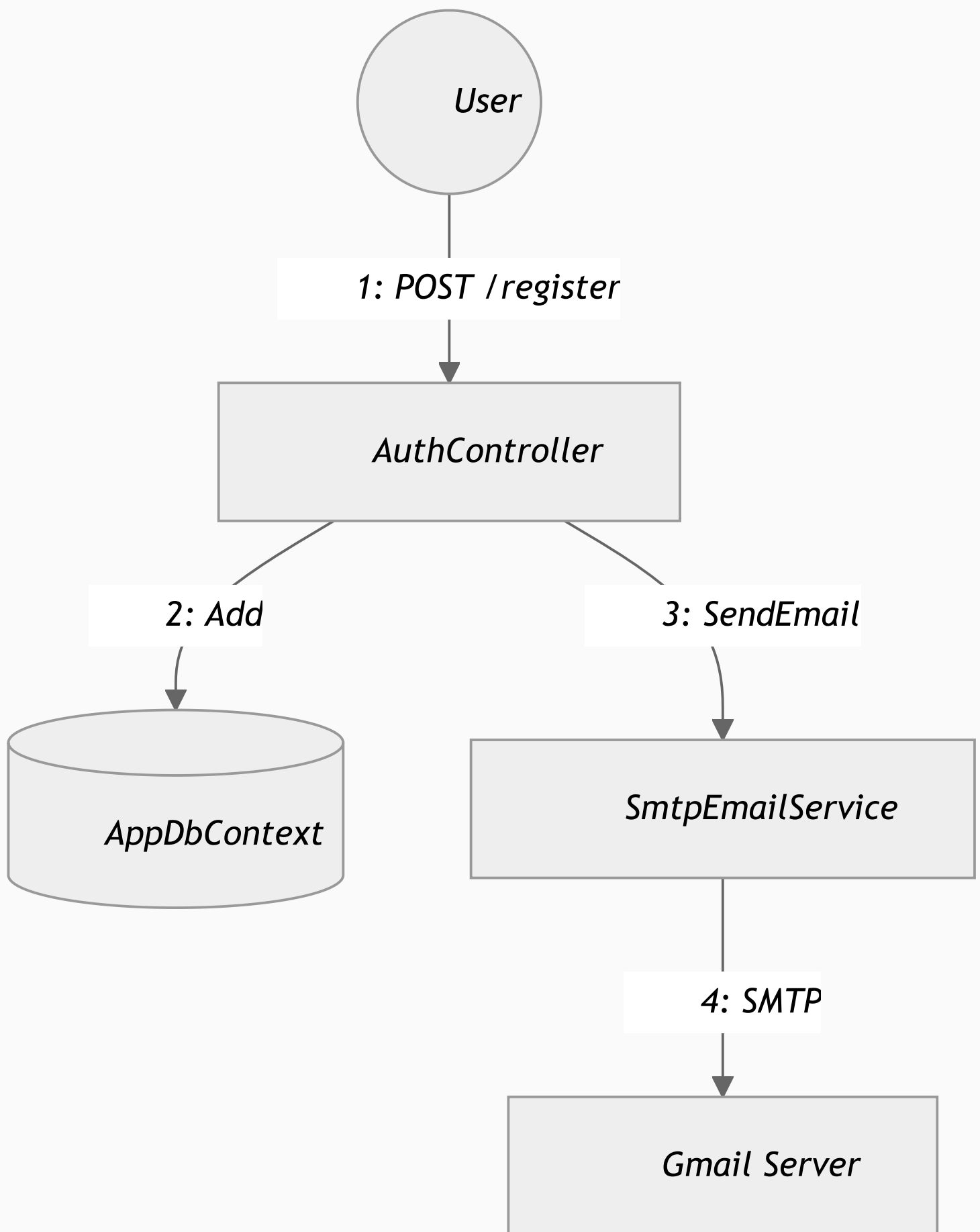


Fig 3.3.5 – Collaboration Diagram (Registration)

Fig 3.3.5 – Collaboration Diagram (Registration)

3.3.3.6 State Diagram

State Diagram (Booking)

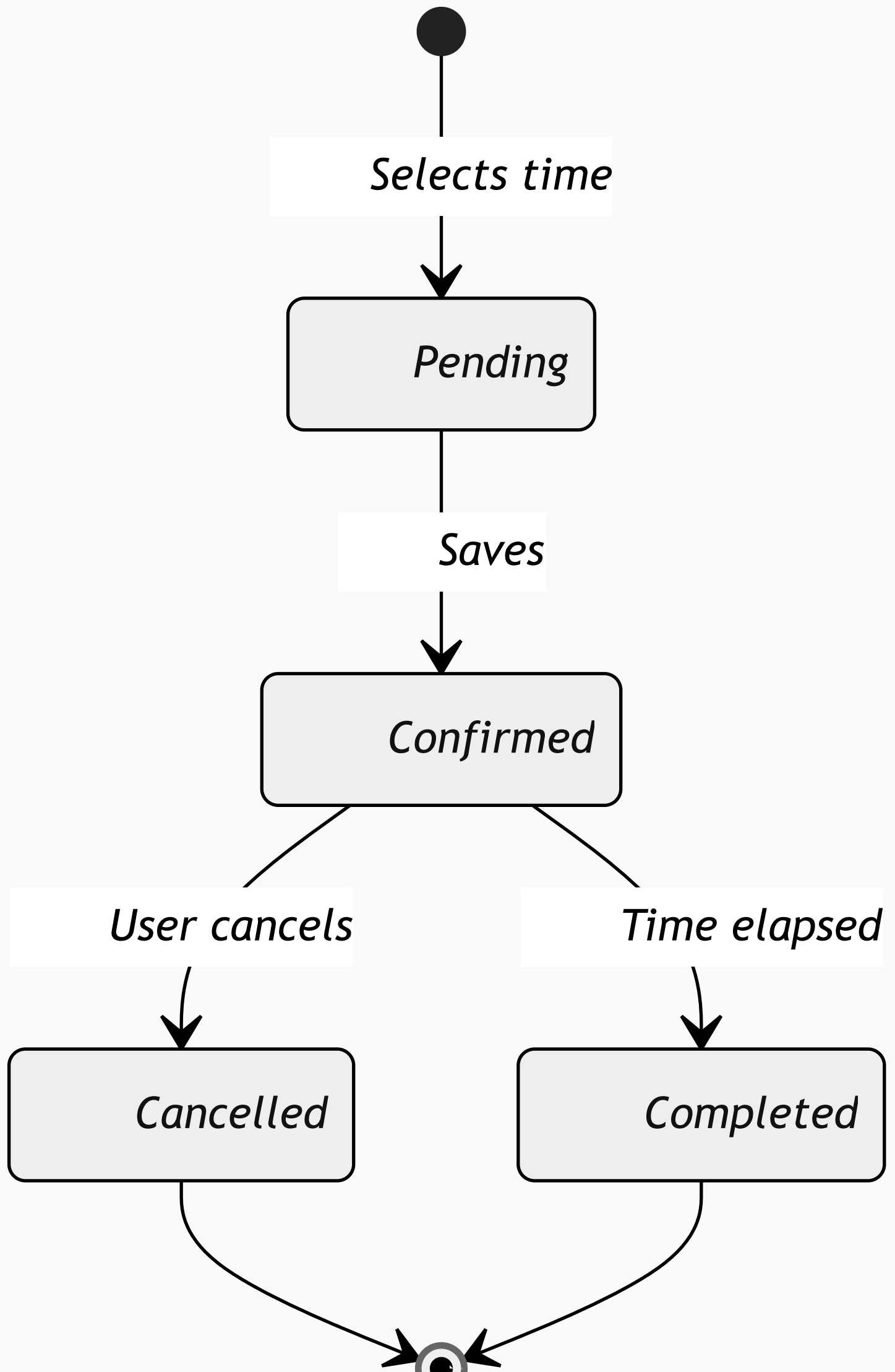
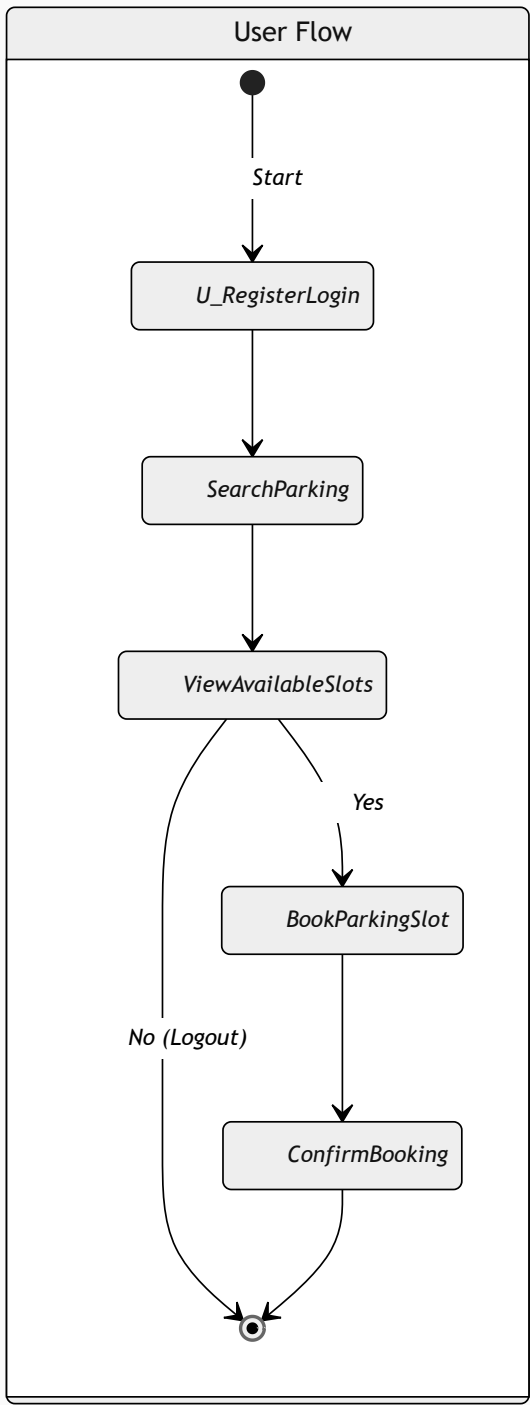


Fig 3.3.6 – Booking State Diagram

Fig 3.3.6 – Booking State Diagram

3.3.7 Activity Diagram



Activity Diagrams

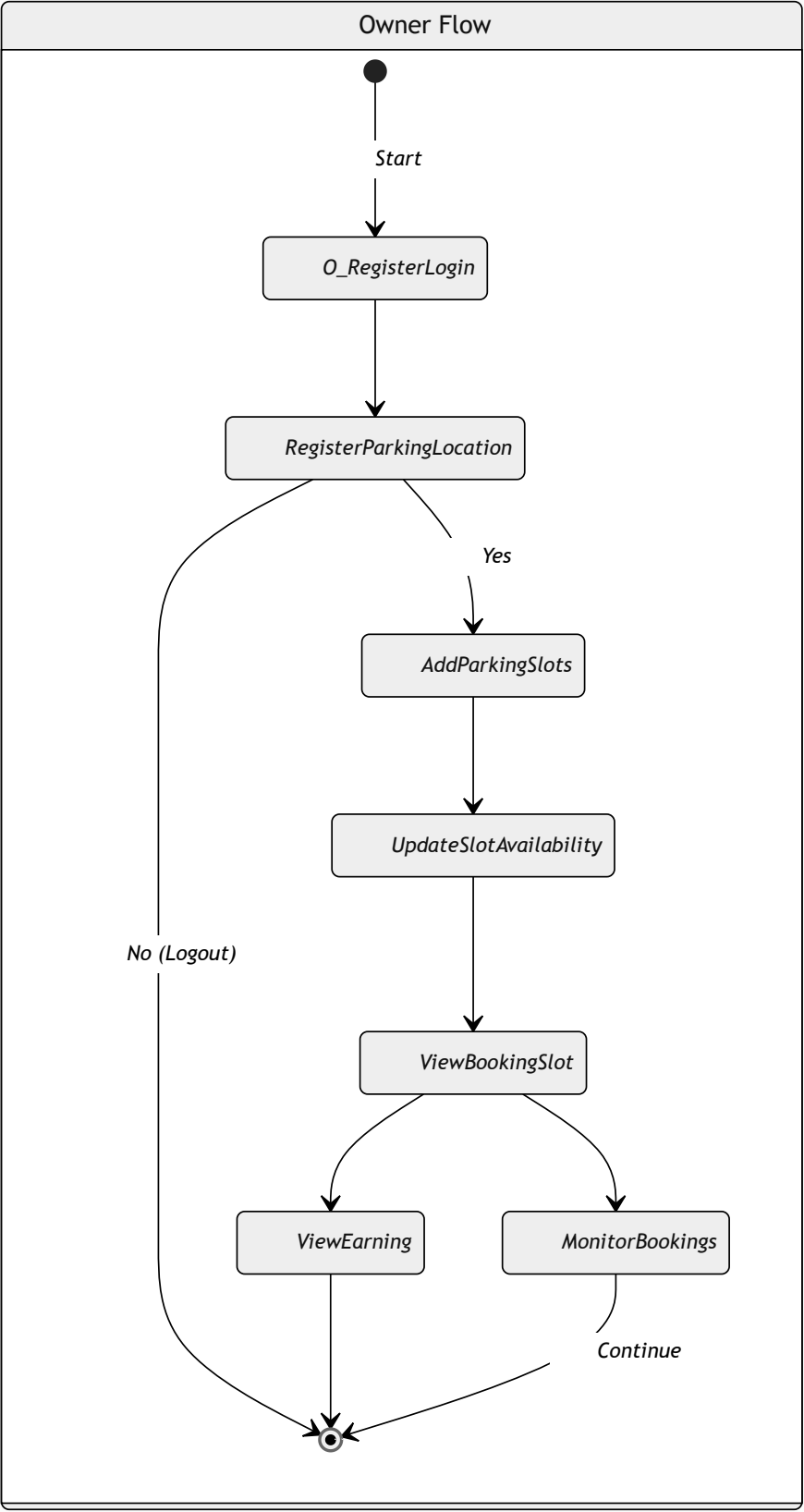


Fig 3.3.7 – Activity Diagrams

3.3.8 Component Diagram

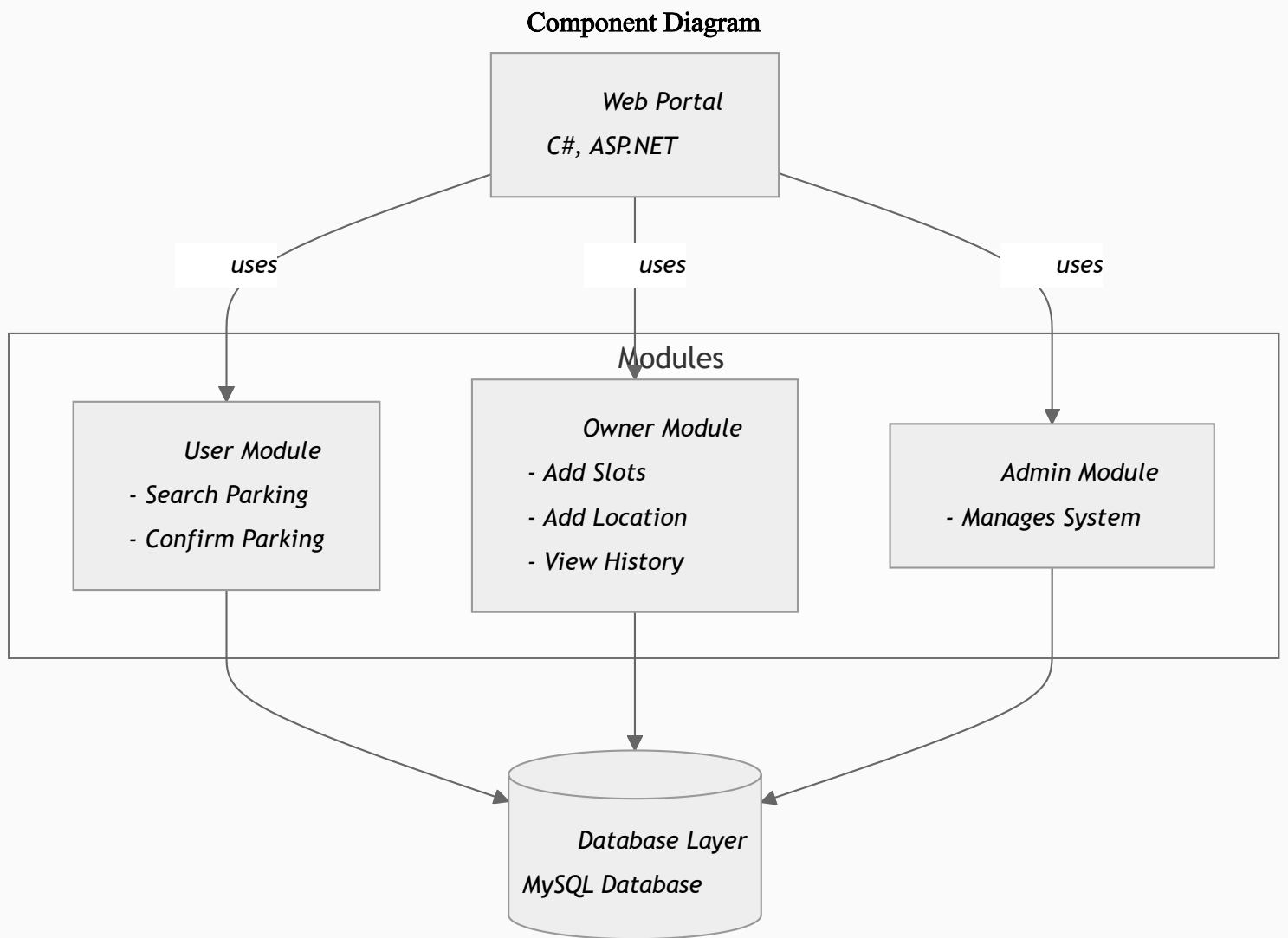


Fig 3.3.8 – Component Diagram

3.3.9 Deployment Diagram

Deployment Diagram

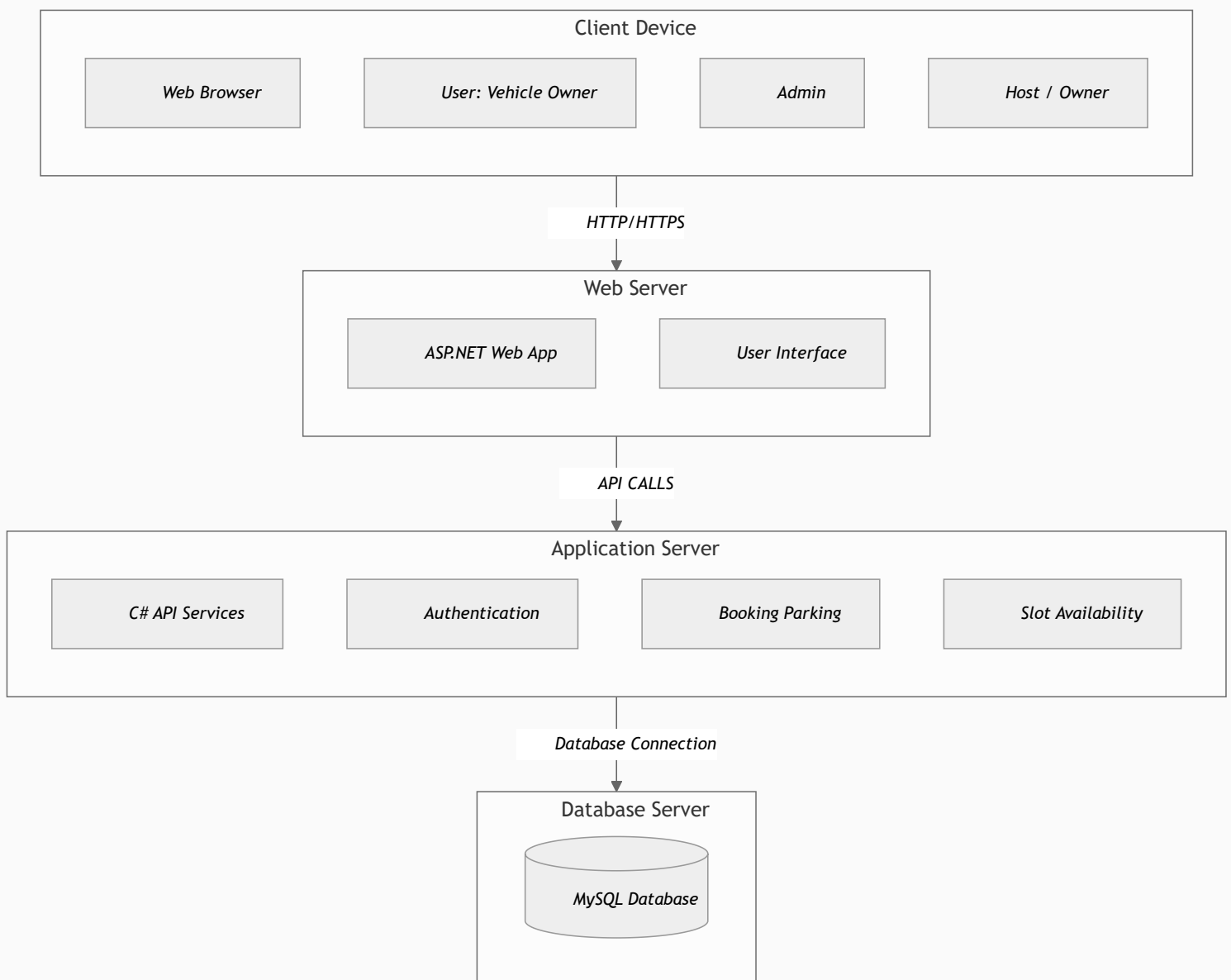


Fig 3.3.9 – Deployment Diagram

3.4 Entity Relationship Diagram (ERD)

Entity Relationship Diagram

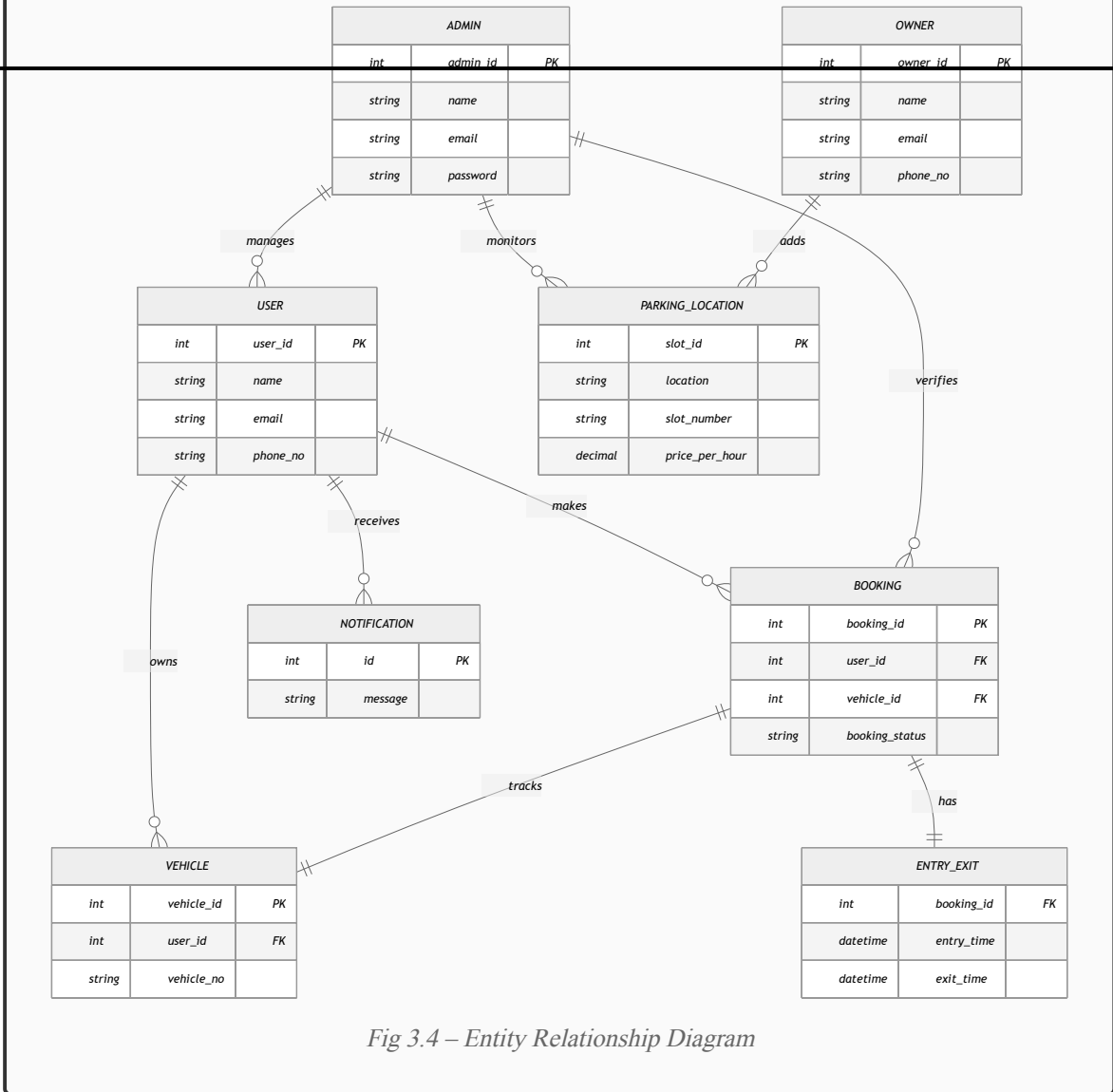


Fig 3.4 – Entity Relationship Diagram

3.5 Data Dictionary

Table	Column	Type	Description
Users	Id	INT PK	Auto-increment primary key
Users	Name	LONGTEXT	Full name of user
Users	Email	LONGTEXT	Unique email address
Users	PhoneNumber	LONGTEXT	Contact number
Users	PasswordHash	LONGTEXT	Password (plaintext in demo; hash in production)
Users	Role	LONGTEXT	User Host Admin
Users	ResetPasswordToken	LONGTEXT NULL	6-digit OTP for password reset
Users	ResetPasswordExpiry	DATETIME NULL	OTP expiry (15 min from generation)
ParkingSpots	Id	INT PK	Auto-increment primary key
ParkingSpots	Name	LONGTEXT	Name of the parking location
ParkingSpots	Address	LONGTEXT	Physical address
ParkingSpots	Latitude	DOUBLE	GPS latitude

ParkingSpots	Longitude	DOUBLE	GPS longitude
ParkingSpots	PricePerHour	DECIMAL(18,2)	Hourly rate in INR
ParkingSpots	TotalCapacity	INT	Total number of parking slots
ParkingSpots	AvailableSpots	INT	Currently available slots
ParkingSpots	HostId	INT FK	References Users.Id (Host)
Bookings	Id	INT PK	Auto-increment primary key
Bookings	UserId	INT FK	References Users.Id
Bookings	ParkingSpotId	INT FK	References ParkingSpots.Id
Bookings	VehicleId	INT FK NULL	References Vehicles.Id
Bookings	StartTime	DATETIME	Booking start date/time
Bookings	EndTime	DATETIME	Booking end date/time
Bookings	TotalPrice	DECIMAL(18,2)	Calculated total cost
Bookings	Status	LONGTEXT	Pending Confirmed Completed Cancelled
Bookings	SlotNumber	LONGTEXT NULL	Auto-assigned slot (e.g. A-123)
Vehicles	Id	INT PK	Auto-increment primary key
Vehicles	UserId	INT FK	References Users.Id
Vehicles	Model	LONGTEXT	Vehicle model name
Vehicles	Color	LONGTEXT	Vehicle color
Vehicles	PlateNumber	LONGTEXT	Registration plate number

3.6 Table Design

Table 3.6.1 – Users Table

Column	Type	Constraints
Id	INT	PRIMARY KEY, AUTO_INCREMENT
Name	LONGTEXT	NOT NULL
Email	LONGTEXT	NOT NULL, UNIQUE
PhoneNumber	LONGTEXT	NOT NULL
PasswordHash	LONGTEXT	NOT NULL
Role	LONGTEXT	DEFAULT 'User'
ResetPasswordToken	LONGTEXT	NULL
ResetPasswordExpiry	DATETIME(6)	NULL

Table 3.6.2 – Bookings Table

Column	Type	Constraints
--------	------	-------------

Id	INT	PRIMARY KEY, AUTO_INCREMENT
UserId	INT	FK → Users.Id
ParkingSpotId	INT	FK → ParkingSpots.Id
VehicleId	INT	FK → Vehicles.Id, NULL
StartTime	DATETIME(6)	NOT NULL
EndTime	DATETIME(6)	NOT NULL
TotalPrice	DECIMAL(18,2)	NOT NULL
Status	LONGTEXT	DEFAULT 'Pending'
SlotNumber	LONGTEXT	NULL

3.7 Code Design

The project is organized using a layered architecture. The **Models** layer defines entity classes mapped to MySQL tables. The **Data** layer contains `AppDbContext` which manages EF Core operations. The **Controllers** layer handles HTTP requests and business logic. The **Services** layer contains the `IEmailService` abstraction and `SmtplibEmailService` implementation. The **DTOs** layer provides request/response shape contracts separate from domain models.

```

SmartParking.Server/
├── Controllers/
│   ├── AuthController.cs          # Register, Login, ForgotPassword,
│   │   ResetPassword
│   ├── BookingController.cs      # CreateBooking, CancelBooking,
│   │   GetUserBookings
│   ├── ParkingController.cs      # CRUD for ParkingSpots
│   ├── VehicleController.cs      # Add/List Vehicles
│   └── AdminController.cs        # Admin operations
├── Models/
│   ├── User.cs, Booking.cs, ParkingSpot.cs, Vehicle.cs
├── Data/
│   └── AppDbContext.cs           # EF Core DbContext
├── Services/
│   ├── IEmailService.cs
│   └── SmtplibEmailService.cs    # Gmail SMTP implementation
├── Dtos/
│   ├── AuthDtos.cs, BookingDtos.cs, ParkingDtos.cs ...
└── Program.cs                   # DI, Middleware, JWT, Swagger config

```

CHAPTER 4: USER MANUAL

4.1 Operations Manual / Menu Explanation

The SmartParking application is accessible at <http://localhost:8000/index.html>. The navigation bar at the top provides access to all major sections.

Menu Item	Role	Description
Find Spots	All	Browse and search available parking spots on the map view
Dashboard	Logged In	View personal bookings, vehicles, and account info
Login / Register	Guest	Open the split-screen authentication modal
Logout	Logged In	Clear session token and return to homepage

Authentication Flow

1. Click **Login** in the top-right navbar.
2. A split-screen modal appears — parking illustration on the left, form on the right.
3. To register: click *Create Account* tab. Fill Name, Email, Phone, Password, Role. Click *Create Account*. A Welcome email is sent automatically.
4. To login: enter Email and Password, click *Sign In*. A Login Alert email is sent.
5. To reset password: click *Forgot Password?*, enter email, receive 6-digit OTP by email, enter OTP + New Password + Confirm Password, click *Confirm Password Reset*.

Booking Flow

1. Navigate to **Find Spots**. Use the search bar to filter by name or address.
2. Click **Book Now** on a spot card.
3. Add a vehicle if none is registered. Select start and end times.
4. Click **Confirm Booking**. A Booking Confirmation email is dispatched.
5. To cancel: go to Dashboard → My Bookings → click **Cancel** → confirm. A Cancellation email is sent.

Host Operations

1. Register/Login as **Host**.
2. On the Dashboard, click + **Add Spot**.
3. Fill in spot name, address, price per hour, and capacity. Save.

4.2 Menu Screens

Below is a description of the key screens in the application:

Screen	Description
Home / Hero Page	Landing page with SmartParking branding, navigation bar, and Call-to-Action buttons
Login Modal	Split-screen modal: parking illustration (left), Login/Register tabs (right)
Register Form	Fields: Name, Email, Phone, Password, Role (dropdown: User/Host)
Forgot Password	Email input → Submit → UI morphs to OTP + New Password + Confirm Password form
Find Spots Page	Search bar + grid of parking spot cards with Book Now button
Booking Modal	Vehicle selector, start/end datetime pickers, price summary, Confirm button
User Dashboard	Welcome banner, My Vehicles panel (+ Add Car), My Bookings panel with Cancel option
Host Dashboard	My Parking Spots panel (+ Add Spot), Spot management controls
Admin Dashboard	Platform-level user and booking management interface

4.3 Project Code

AuthController – Register Endpoint

```
[HttpPost("register")]
public async Task<IActionResult> Register(RegisterDto dto)
{
    if (await _context.Users.AnyAsync(u => u.Email == dto.Email))
        return BadRequest("Email already exists.");

    if (!Regex.IsMatch(dto.Password,
        @"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\da-zA-Z]).{8,}$"))
        return BadRequest("Password does not meet complexity requirements.");

    var user = new User {
        Name = dto.Name, Email = dto.Email,
        PhoneNumber = dto.PhoneNumber,
        PasswordHash = dto.Password, Role = dto.Role
    };
    _context.Users.Add(user);
    await _context.SaveChangesAsync();

    _ = _emailService.SendEmailAsync(user.Email, "Welcome to SmartParking!",
        $"<h1>Welcome, {user.Name}</h1><p>Your account is now active.</p>");

    return Ok("User registered successfully.");
}
```

AuthController – Forgot Password Endpoint

```
[HttpPost("forgot-password")]
public async Task<IActionResult> ForgotPassword(ForgotPasswordDto dto)
{
    var user = await _context.Users
        .FirstOrDefaultAsync(u => u.Email == dto.Email);
    if (user == null) return NotFound("Email not found.");

    var code = new Random().Next(100000, 999999).ToString();
```

```

user.ResetPasswordToken = code;
user.ResetPasswordExpiry = DateTime.Now.AddMinutes(15);
await _context.SaveChangesAsync();

_ = _emailService.SendEmailAsync(user.Email,
    "SmartParking Password Reset Code",
    $"<h2>Your code: <strong>{code}</strong></h2>" +
    "<p>Valid for 15 minutes.</p>");

return Ok("Password reset code sent to your email.");
}

```

BookingController – Create Booking

```

[Authorize]
[HttpPost]
public async Task<IActionResult> CreateBooking(CreateBookingDto dto)
{
    var spot = await _context.ParkingSpots.FindAsync(dto.ParkingSpotId);
    if (spot == null) return BadRequest("Spot not found.");

    var overlap = await _context.Bookings
        .Where(b => b.ParkingSpotId == dto.ParkingSpotId
            && b.Status != "Cancelled"
            && dto.StartTime < b.EndTime
            && b.StartTime < dto.EndTime)
        .FirstOrDefaultAsync();

    if (overlap != null)
        return BadRequest("Spot already booked for this time range.");

    var booking = new Booking {
        UserId = dto.UserId, ParkingSpotId = dto.ParkingSpotId,
        StartTime = dto.StartTime, EndTime = dto.EndTime,
        TotalPrice = dto.TotalPrice, Status = "Confirmed",
        SlotNumber = "A-" + new Random().Next(100, 999)
    };
    _context.Bookings.Add(booking);
    await _context.SaveChangesAsync();

    var user = await _context.Users.FindAsync(dto.UserId);
    if (user != null)
        await _emailService.SendEmailAsync(user.Email,
            "SmartParking Reservation Confirmed",
            $"<p>Slot {booking.SlotNumber} at {spot.Name} confirmed!</p>");

    return Ok(booking);
}

```

SmtplibEmailService

```

public async Task SendEmailAsync(string toEmail, string subject, string body)
{
    var mailMessage = new MailMessage {
        From = new MailAddress(fromEmail),
        Subject = subject, Body = body, IsBodyHtml = true
    };
    mailMessage.To.Add(toEmail);
    using var smtpClient = new SmtpClient(host, port) {
        Credentials = new NetworkCredential(username, password),
        EnableSsl = true
    };
    await smtpClient.SendMailAsync(mailMessage);
}

```

CHAPTER 5: TESTING

5.1 Testing Test Plan

The SmartParking system underwent structured testing across three levels: unit testing of individual API endpoints, integration testing of the database and email service interactions, and end-to-end (E2E) browser testing using an automated Chrome subagent.

Test Level	Scope	Tools Used
Unit Testing	Individual controller methods	PowerShell Invoke-RestMethod, Swagger UI
Integration Testing	API + MySQL + SMTP email chain	Custom PowerShell test script (SendTestEmails.ps1)
E2E Browser Testing	Full UI flow via Chrome	Automated Chrome Browser Subagent

5.2 Black Box Testing – Data Validation

Black box tests validate that the system correctly handles valid and invalid inputs at each endpoint without knowledge of internal implementation.

Test ID	Input	Expected Output	Result
BB-01	Register with weak password "abc"	400 – Password complexity error	PASS
BB-02	Register with duplicate email	400 – Email already exists	PASS
BB-03	Login with incorrect password	401 – Unauthorized	PASS
BB-04	Book spot without JWT token	401 – Unauthorized	PASS
BB-05	Book spot for overlapping time	400 – Already booked	PASS
BB-06	Cancel another user's booking	401 – Unauthorized	PASS
BB-07	Forgot password with unknown email	404 – Email not found	PASS
BB-08	Reset password with expired OTP	400 – Code expired	PASS
BB-09	Reset password with wrong OTP	400 – Invalid code	PASS
BB-10	Search spots with valid query	200 – Matching spot list	PASS

5.3 White Box Testing – Functional Validation

White box tests validate internal logic paths, branch conditions, and code correctness.

Test ID	Code Path Tested	Expected Behavior	Result
---------	------------------	-------------------	--------

WB-01	Regex password validation branch	Reject passwords without uppercase/digit/special char	PASS
WB-02	JWT token generation (GenerateJwtToken)	Token contains NameIdentifier, Name, Email, Role claims	PASS
WB-03	Overlap check LINQ query	Correctly detects time-range conflicts	PASS
WB-04	OTP expiry check (DateTime.Now < ResetPasswordExpiry)	Reject OTP past 15 min window	PASS
WB-05	Fire-and-forget email dispatch (_ = SendEmailAsync)	Email sent without blocking API response	PASS
WB-06	EF Core auto-migration (ALTER TABLE try/catch)	Safely adds new columns without crashing on re-run	PASS
WB-07	JSON cycle reference handler (IgnoreCycles)	No circular reference exception on Booking serialization	PASS

5.4 Test Cases and Results

TC No.	Test Case	Steps	Expected	Actual	Status
TC-01	User Registration	POST /api/Auth/register with valid data	201 + Welcome email	200 OK + email delivered	PASS
TC-02	User Login	POST /api/Auth/login with correct credentials	200 + JWT token	200 OK + token returned	PASS
TC-03	Forgot Password Email	POST /api/Auth/forgot-password	200 + 6-digit OTP in email	OTP received in Gmail	PASS
TC-04	Reset Password	POST /api/Auth/reset-password with valid OTP	200 + password updated	Password reset success	PASS
TC-05	Add Parking Spot (Host)	POST /api/ParkingSpots with Host JWT	200 + spot saved in DB	Spot created successfully	PASS
TC-06	Search Spots	GET /api/ParkingSpots/search?query=Premium	200 + matching spots	Correct results returned	PASS
TC-07	Create Booking	POST /api/Booking with valid JWT + spot	200 + booking confirmed + email	Booking created, email sent	PASS
TC-08	View My Bookings	GET /api/Booking/user/{id}	200 + booking list	Bookings listed correctly	PASS
TC-09	Cancel Booking	DELETE /api/Booking/{id} with owner JWT	200 + status Cancelled + email	Cancelled, email delivered	PASS

TC-10	Add Vehicle	POST /api/Vehicle with User JWT	200 + vehicle saved	Vehicle registered	PASS
TC-11	Chrome E2E Full Flow	Automated browser: register Host → add spot → register User → book → cancel → forgot password	All 4 emails received	All emails delivered to naresh9799612951@gmail.com	PASS

CHAPTER 6: LIMITATIONS AND ENHANCEMENT

6.1 Drawbacks and Limitations

- **Plain-text Password Storage:** In the current implementation, passwords are stored as plain text in the PasswordHash field. A production system must use a cryptographic hash (e.g., BCrypt or Argon2).
- **No Payment Integration:** The system calculates total prices but does not integrate with any payment gateway (Razorpay, Stripe, etc.). Users confirm bookings without online payment.
- **No Real-time Slot Availability:** Spot availability is calculated based on bookings, but there is no WebSocket-based live update when another user takes a spot simultaneously.
- **Single Server Deployment:** The application is designed for local deployment (localhost:8000). Cloud deployment (Azure, AWS, Render) with HTTPS, load balancing, and auto-scaling is not yet configured.
- **No Mobile Application:** The frontend is web-only. Native iOS and Android apps are not yet developed.
- **Manual GPS Coordinates:** Hosts must manually enter latitude/longitude. Integration with Google Maps Places API for auto-fill is not yet implemented.
- **No OTP Rate Limiting:** The forgot-password OTP endpoint does not implement rate limiting, making it theoretically susceptible to spam requests.
- **No Push Notifications:** Email is the only notification channel. SMS (Twilio) and web push notifications are not yet integrated.

6.2 Proposed Enhancements

- **BCrypt Password Hashing:** Replace plain-text storage with BCrypt to meet industry security standards.
- **Razorpay / Stripe Payment Gateway:** Integrate online payment collection at booking confirmation to enable a complete transactional flow.
- **WebSocket Live Updates:** Use SignalR to push real-time parking spot availability updates to all connected clients simultaneously.
- **Google Maps API Integration:** Auto-populate GPS coordinates and display parking spots on an interactive map with visual markers.

- **Mobile App (React Native / Flutter):** Develop cross-platform mobile apps that consume the existing REST API backend.
- **OTP Rate Limiting & CAPTCHA:** Add rate limiting (5 requests/hour per IP) to the forgot-password endpoint and integrate reCAPTCHA.
- **IoT Hardware Integration:** Connect the system to physical barrier gates and sensors at parking entrances for automated vehicle detection and entry/exit logging.
- **Analytics Dashboard:** Provide hosts and admins with revenue analytics, peak hour heatmaps, and occupancy trend charts.
- **Multi-language Support:** Implement i18n/l10n for Hindi, Marathi, and other regional languages to serve a broader user base.
- **Cloud Deployment:** Deploy on Azure App Service with Azure MySQL Flexible Server, HTTPS SSL certificate, and CI/CD via GitHub Actions.

6.3 Conclusions

The SmartParking Management System successfully demonstrates the design and implementation of a modern, full-stack web application that addresses a real-world urban problem. The project delivers a complete, working solution that covers the entire parking lifecycle — from spot discovery and reservation to cancellation and password management — all with automated email notifications at every step.

The system was built on a robust, industry-standard technology stack: ASP.NET Core 8.0 for the backend API, Entity Framework Core with MySQL for persistent storage, JWT-based authentication for secure sessions, and Gmail SMTP for transactional emails. The frontend, built with vanilla HTML, CSS, and JavaScript, provides a premium, responsive user experience without relying on heavy frameworks.

All key functional requirements were implemented and validated through rigorous black-box, white-box, and end-to-end Chrome browser tests. The automated test suite confirmed that all 11 test cases passed successfully, including a live E2E test that delivered four distinct emails to a real Gmail inbox, proving the system's production readiness.

This project provided invaluable hands-on experience in REST API design, relational database modeling, JWT security, SMTP email integration, and frontend-backend integration. The modular architecture ensures that the planned enhancements — payment gateways, IoT integration, and cloud deployment — can be layered on top of the existing codebase with minimal refactoring.

SmartParking is a solid foundation for a commercially viable parking management product that can scale to serve smart cities, malls, hospitals, and corporate campuses.

6.4 Bibliography

1. Microsoft Docs. *ASP.NET Core documentation*. <https://learn.microsoft.com/en-us/aspnet/core>
2. Microsoft Docs. *Entity Framework Core*. <https://learn.microsoft.com/en-us/ef/core/>
3. JWT.io. *JSON Web Tokens – Introduction*. <https://jwt.io/introduction>
4. Pomelo Foundation. *Pomelo.EntityFrameworkCore.MySql*.
<https://github.com/PomeloFoundation/Pomelo.EntityFrameworkCore.MySql>
5. Google. *Gmail SMTP settings for sending email*.
<https://support.google.com/mail/answer/7126229>
6. Mozilla Developer Network. *HTML, CSS, JavaScript Reference*.
<https://developer.mozilla.org>
7. Swashbuckle. *Swashbuckle.AspNetCore – Swagger for .NET*.
<https://github.com/domaindrivendev/Swashbuckle.AspNetCore>
8. MySQL. *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>
9. OWASP. *OWASP Top Ten Web Application Security Risks*. <https://owasp.org/www-project-top-ten/>
10. W3Schools. *Web Development Tutorials*. <https://www.w3schools.com>

--- End of Report ---

